

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа

Выполнил:

Милютин Никита Александрович

К3339

Проверил:

Добряков Д. И.

Санкт-Петербург

2022 г.

Задача

Опишите все функциональные и нефункциональные требования к вашему бэкенд-приложению

Выберите формат реализации API, которого вы будете придерживаться: REST API или Backend For Frontend

Спроектируйте все эндпоинты вашего API в формате OpenAPI-схемы с учётом реализованной диаграммы БД

Опишите тело каждого запроса и тело каждого ответа, продумайте возможные ошибки, возвращаемые из API

Ход работы

1. Функциональные требования

Бэкенд-приложение должно обеспечивать следующий функционал:

1. Регистрация пользователя.
2. Авторизация пользователя.
3. Получение данных о текущем пользователе.
4. Просмотр и редактирование профиля пользователя.
5. Поиск ресторанов с фильтрацией по кухне, расположению и ценовой категории.
6. Получение полной информации о конкретном ресторане.
7. Получение фотографий ресторана.
8. Получение файла меню ресторана.
9. Получение отзывов о ресторане.
10. Создание бронирования столика.
11. Просмотр истории бронирований пользователя.
12. Отмена бронирования.
13. Получение списка столиков ресторана.
14. Добавление отзыва о ресторане.
15. Возможность загрузки рестораном файла меню.
16. Возможность загрузки фотографий ресторана.

2. Нефункциональные требования

К разрабатываемому приложению предъявляются следующие нефункциональные требования:

1. API должно быть построено по единым правилам именования маршрутов и форматов данных.
2. Обмен данными между клиентом и сервером должен происходить в формате JSON.
3. API должно поддерживать аутентификацию и авторизацию пользователей.
4. Пароли пользователей не должны храниться в открытом виде.
5. Время ответа сервера на стандартные запросы должно быть минимальным.
6. Система должна обеспечивать защиту от некорректных входных данных.
7. API должно быть масштабируемым и пригодным для дальнейшего расширения.
8. Все ошибки должны возвращаться в едином формате.
9. Приложение должно обеспечивать целостность данных при создании бронирований.
10. Должна быть обеспечена возможность интеграции с веб- или мобильным клиентом.

3. Выбор формата API

Для реализации был выбран формат **REST API**.

Выбор REST API обусловлен следующими причинами:

- простота проектирования и реализации;
- удобная работа с HTTP-методами;
- понятная структура маршрутов;
- широкая поддержка со стороны фронтенд-приложений и мобильных клиентов;
- удобство документирования через OpenAPI.

4. Основные сущности API

В API используются следующие основные сущности:

- User
- Restaurant
- Cuisine
- Table
- Reservation
- Review
- RestaurantPhoto
- RestaurantMenu

5. Общая логика работы API

API строится по ресурсному принципу.

Каждый эндпоинт отвечает за работу с определённой сущностью.

Для чтения данных используются методы **GET**, для создания — **POST**, для изменения — **PUT** или **PATCH**, для удаления или отмены — **DELETE**.

После этого были спроектированы основные эндпоинты системы, связанные с регистрацией и авторизацией пользователей, работой с ресторанами, бронированиями, отзывами, фотографиями и файлами меню. Для каждого эндпоинта были продуманы структура запросов, структура ответов и возможные варианты ошибок. Далее была составлена OpenAPI-схема, описывающая все разработанные маршруты API, и подготовлена основа для её локального запуска через Express и TypeScript.

Вывод

В результате выполнения работы был спроектирован API для приложения бронирования столиков в ресторанах. Были сформулированы основные требования к системе, выбран подход REST API и описаны ключевые маршруты взаимодействия клиента с сервером. Также была подготовлена OpenAPI-схема, которая может использоваться как основа для дальнейшей разработки и тестирования бэкенд-приложения. Выполненная работа позволила получить практический опыт проектирования серверного API и подготовки технической документации.